Calculating Correct and Efficient Programs in Rocq

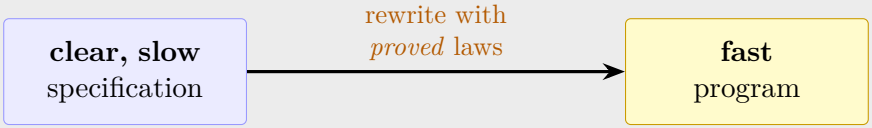
David Castro-Perez

David.CastroPerez@rhul.ac.uk



Based on joint work with Marco Paviotti and Michael Vollmer (University of Kent):
Program Optimisations via Hylomorphisms for Extraction of Executable Code, ITP 2025.

In One Slide



The idea (1980s):

- Programs as algebra
- Optimise by rewriting
- Correct by construction

Our work:

- Every step machine-checked
- No extra axioms
- Runnable code out

The Problem

Fast *and* Correct

A Specification and a Program



A Specification and a Program




```
let rec sort_times_two = function
  | [] -> []
  | h :: t ->
    let (l, r) = partition (fun x -> x < h) t in
    sort_times_two l @ (h * 2) :: sort_times_two r
```

A Specification and a Program




```
let rec sort_times_two = function
  | [] -> []
  | h :: t ->
    let (l, r) = partition (fun x -> x < h) t in
    sort_times_two l @ (h * 2) :: sort_times_two r
```

Are they equal? Can we *derive* the second from the first?

Three Ways to Fast and Correct Code

Approach	Good	But
Write it, then **verify** it	any program	a new proof for every program

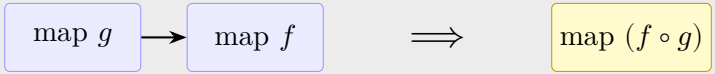
Three Ways to Fast and Correct Code

Approach	**Good**	**But**
Write it, then **verify** it	any program	a new proof for every program
Verified compiler	translation is correct	keeps your algorithm: $O(n^3)$ stays $O(n^3)$

Three Ways to Fast and Correct Code

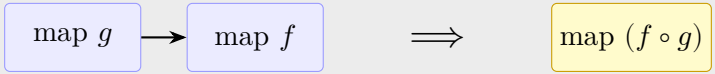
Approach	**Good**	**But**
Write it, then **verify** it	any program	a new proof for every program
Verified compiler	translation is correct	keeps your algorithm: $O(n^3)$ stays $O(n^3)$
Calculate it from a spec	proof comes with the derivation	needs laws that *apply*

You Already Trust Rewrite Rules



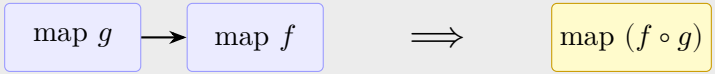
- ML compilers *fuse* tensor operations into one kernel.

You Already Trust Rewrite Rules



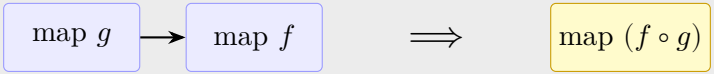
- ML compilers *fuse* tensor operations into one kernel.
- Database engines rewrite query plans.

You Already Trust Rewrite Rules



- ML compilers *fuse* tensor operations into one kernel.
- Database engines rewrite query plans.
- Compilers inline, fuse loops, remove temporary data.

You Already Trust Rewrite Rules

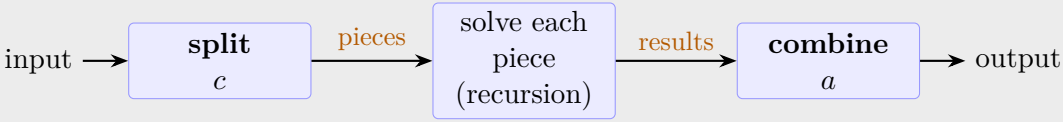


- ML compilers *fuse* tensor operations into one kernel.
- Database engines rewrite query plans.
- Compilers inline, fuse loops, remove temporary data.
- Usually: a **fixed** set of rules, **trusted**, not proved.

The Laws

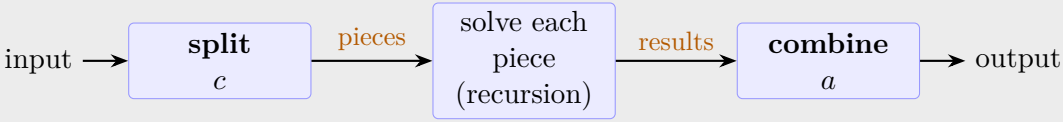
Divide and Conquer as Algebra

One Shape for Many Algorithms



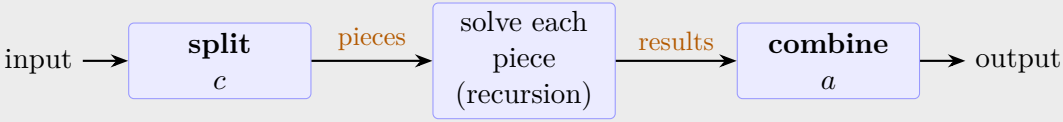
Algorithm	**split** c	**combine** a

One Shape for Many Algorithms



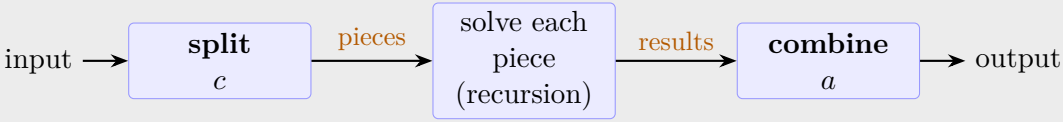
Algorithm	**split** c	**combine** a
quicksort	partition around a pivot	concatenate

One Shape for Many Algorithms



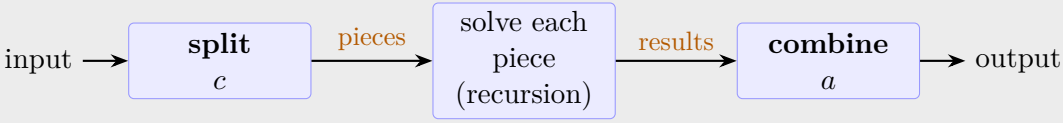
Algorithm	**split** c	**combine** a
quicksort	partition around a pivot	concatenate
mergesort	cut in half	merge

One Shape for Many Algorithms



Algorithm	**split** c	**combine** a
quicksort	partition around a pivot	concatenate
mergesort	cut in half	merge
x^n	$n \mapsto$ last bit, $n/2$	square, maybe multiply by x

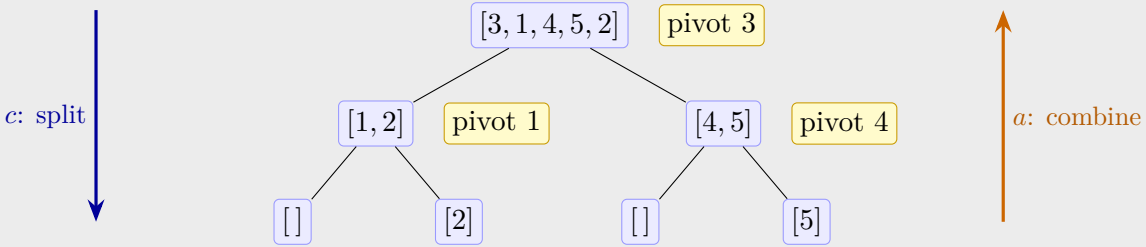
One Shape for Many Algorithms



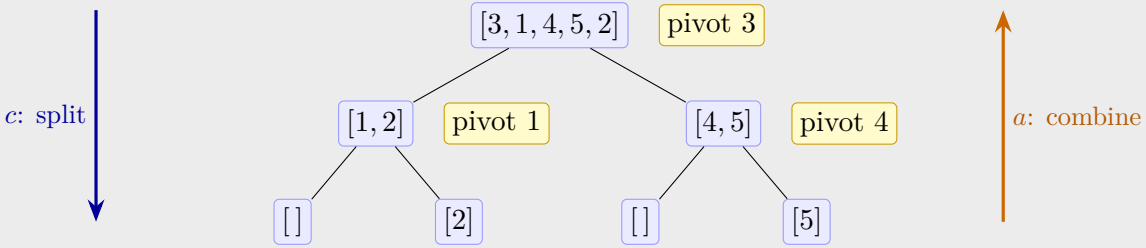
Algorithm	**split** c	**combine** a
quicksort	partition around a pivot	concatenate
mergesort	cut in half	merge
x^n	$n \mapsto$ last bit, $n/2$	square, maybe multiply by x

Hylomorphism $[\![a, c]\!]$: the recursion is *generic*; only a and c change.

The Call Tree of Quicksort

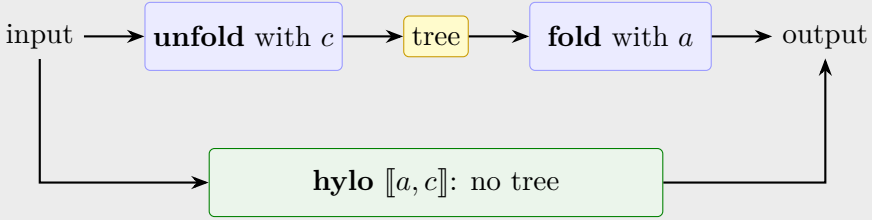


The Call Tree of Quicksort



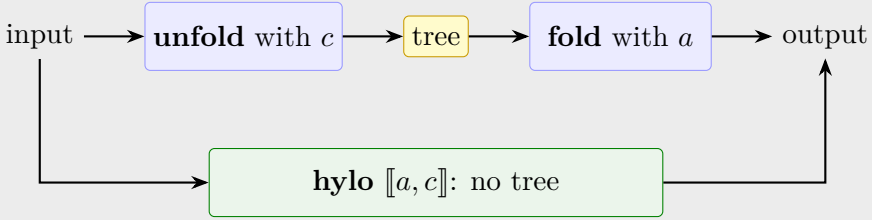
The tree is *virtual*: it is never built in memory.

Producers and Consumers



- Specification: *build*, then *consume*. Easy to read.

Producers and Consumers



- Specification: *build*, then *consume*. Easy to read.
- Program: do both at once. Fast.

Two Laws

Fusion $[\![a, c]\!] \rightarrow h = [\![b, c]\!]$

if h distributes over a

Deforestation $[\![\mathsf{build}, c]\!] \rightarrow [\![a, \mathsf{take_apart}]\!] = [\![a, c]\!]$

- Proved **once**, for every split, combine, data type.

Two Laws

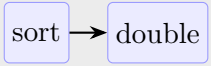
Fusion $[\![a, c]\!] \rightarrow h = [\![b, c]\!]$

if h distributes over a

Deforestation $[\![\mathsf{build}, c]\!] \rightarrow [\![a, \mathsf{take_apart}]\!] = [\![a, c]\!]$

- Proved **once**, for every split, combine, data type.
- Each use is a *rewrite step*.

Deriving the Example



Deriving the Example

$$\mathrm{sort} \rightarrow \mathrm{double}$$

$$= \quad [\![\mathsf{concat}, \mathsf{partition}]\!] \rightarrow \mathrm{double} \qquad \text{definition of quicksort}$$

Deriving the Example

$$\mathsf{sort} \rightarrow \mathsf{double}$$

$$= \quad [\![\mathsf{concat}, \mathsf{partition}]\!] \rightarrow \mathsf{double} \qquad \text{definition of quicksort}$$

$$= \quad [\![\mathsf{concat}\ \text{and double pivot}, \mathsf{partition}]\!] \qquad \text{fusion}$$

Deriving the Example

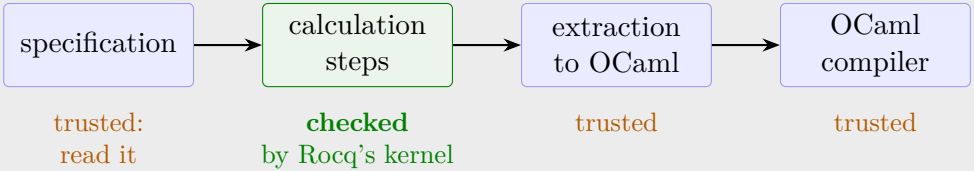
$$\begin{aligned}
&\boxed{\text{sort}} \rightarrow \boxed{\text{double}} \\
=\ &\boxed{[\![\mathsf{concat}, \mathsf{partition}]\!]} \rightarrow \boxed{\text{double}} && \text{definition of quicksort} \\
=\ &\boxed{[\![\mathsf{concat}\text{ and double pivot}, \mathsf{partition}]\!]} && \text{fusion}
\end{aligned}$$

Each $=$ is a **theorem** applied by a proof assistant.

Machine-Checked

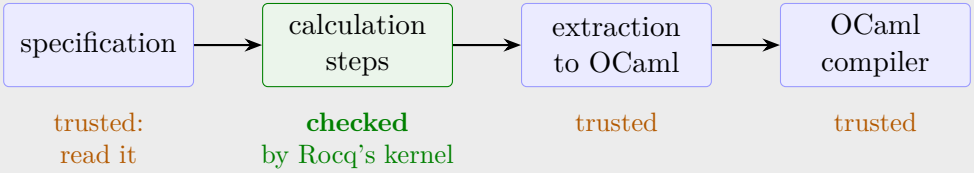
Hylomorphisms in Rocq

What You Have to Trust



- Who picked the steps does not matter: a person, a script, a model.

What You Have to Trust



- Who picked the steps does not matter: a person, a script, a model.
- A wrong step is **rejected**, not miscompiled.

Three Obstacles

Termination

every function must provably stop

No extra axioms

comparing functions needs one

Clean code

generic proofs extract to unsafe casts

Three Obstacles

Termination

every function must provably stop

No extra axioms

comparing functions needs one

Clean code

generic proofs extract to unsafe casts

Haskell has the laws but not termination; Rocq has termination but rejects the textbook definitions.

What the Textbook Encoding Extracts To

```
type sh = __                                    (* = Obj.t *)
type pos = __
type 'x ext = { shape : sh; cont : (pos -> 'x) }

let merge x =
  let { shape = shape0; cont = k } = x in
  (match Obj.magic shape0 with
   | Some h -> app (Obj.magic k True) (Cons (h, (Obj.magic k False)))
   | None -> Nil)
```

(simplified)

What the Textbook Encoding Extracts To

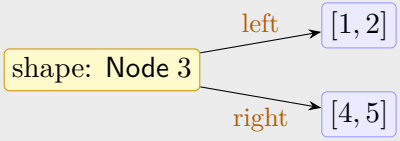
```
type sh = __                                        (* = Obj.t *)
type pos = __
type 'x ext = { shape : sh; cont : (pos -> 'x) }

let merge x =
  let { shape = shape0; cont = k } = x in
  (match Obj.magic shape0 with
   | Some h -> app (Obj.magic k True) (Cons (h, (Obj.magic k False)))
   | None -> Nil)
```

(simplified)

`Obj.magic`: an **unchecked cast**. Unsafe as soon as this code meets other code.

Our Fix: Shapes and Positions

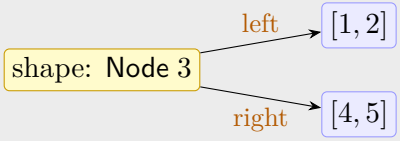


shape: Leaf no positions

valid?	left	right
`Node h`	yes	yes
`Leaf`	no	no

- One type of positions + a **yes/no check**: plain OCaml types.

Our Fix: Shapes and Positions



shape: Leaf no positions

valid?	left	right
`Node h`	yes	yes
`Leaf`	no	no

- One type of positions + a **yes/no check**: plain OCaml types.
- Proofs about the check are **erased** from the code.

Termination: Recursion on a Proof



- Proved **once per split**, e.g. “pieces are shorter”.

Termination: Recursion on a Proof



- Proved **once per split**, e.g. “pieces are shorter”.
- The proof is scaffolding: **erased** at extraction.

No Extra Axioms

```
Print Assumptions hylo_fusion_l.          Closed under the global context
Print Assumptions mss_calc.               Closed under the global context
Print Assumptions qsort_times_two_calc.   Axioms: PrimInt63.int, ltb, mul
```

(simplified)

- Equality = *same results on equal inputs* (setoids).

No Extra Axioms

```
Print Assumptions hylo_fusion_l.          Closed under the global context
Print Assumptions mss_calc.               Closed under the global context
Print Assumptions qsort_times_two_calc.   Axioms: PrimInt63.int, ltb, mul
```

(simplified)

- Equality = *same results on equal inputs* (setoids).
- Laws become `rewrite` steps.

In Practice

From Calculation to Code

A Calculation in Rocq

```
Derive qsort_times_two
  in (Lmap times_two \o cata merge \o rana tsplit =e qsort_times_two)
  as qsort_times_two_calc.
Proof.
  rewrite map_merge_fusion, hylo_ana, deforest by apply l_out_in.
  finished.
Qed.
```

A Calculation in Rocq




```
Derive qsort_times_two
  in (Lmap times_two \o cata merge \o rana tsplit =e qsort_times_two)
  as qsort_times_two_calc.
Proof.
  rewrite map_merge_fusion, hylo_ana, deforest by apply l_out_in.
  finished.
Qed.
```

A Calculation in Rocq

```
Derive qsort_times_two
  in (Lmap times_two \o cata merge \o rana tsplit =e qsort_times_two)
  as qsort_times_two_calc.
Proof.
  rewrite map_merge_fusion, hylo_ana, deforest by apply l_out_in.
  finished.
Qed.
```

specification

the laws

What Comes Out

```
let rec qsort_times_two_code = function
| [] -> []
| h :: t ->
  let (l, r) = partition (fun x0 -> ltb x0 h) t in
  let x0 = fun e ->
    qsort_times_two_code (match e with Lbranch -> l | Rbranch -> r) in
  app (x0 Lbranch) ((mul (Uint63.of_int (2)) h) :: (x0 Rbranch))
```

(simplified)

What Comes Out

```
let rec qsort_times_two_code = function
| [] -> []
| h :: t ->
  let (l, r) = partition (fun x0 -> ltb x0 h) t in
  let x0 = fun e ->
    qsort_times_two_code (match e with Lbranch -> l | Rbranch -> r) in
  app (x0 Lbranch) ((mul (Uint63.of_int (2)) h) :: (x0 Rbranch))
```

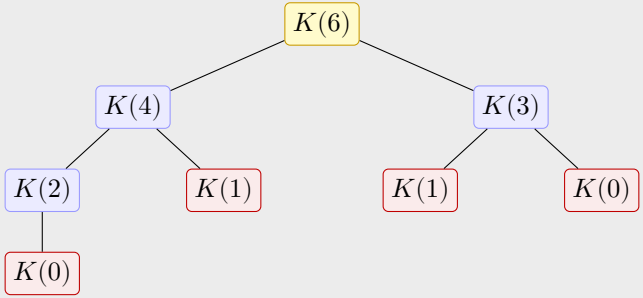
(simplified)

Hand-written (start of the talk):

```
let rec sort_times_two = function
  | [] -> []
  | h :: t ->
    let (l, r) = partition (fun x -> x < h) t in
    sort_times_two l @ (h * 2) :: sort_times_two r
```

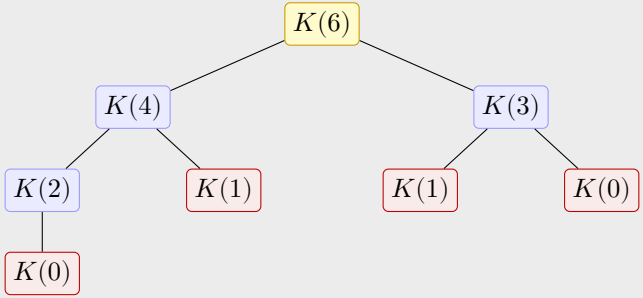
Dynamic Programming: Knapsack

$$K(0) = 0 \qquad K(c) = \max\left(\{0\} \cup \{\, K(c - w_i) + v_i \mid 0 < w_i \leq c \,\}\right)$$



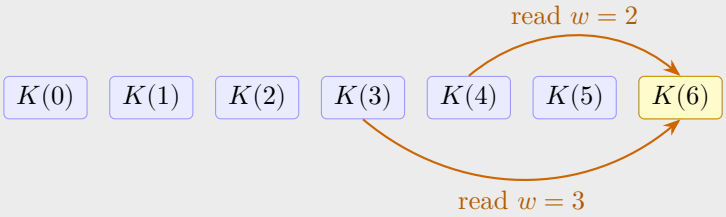
Dynamic Programming: Knapsack

$$K(0) = 0 \qquad K(c) = \max\left(\{0\} \cup \{\, K(c - w_i) + v_i \mid 0 < w_i \leq c \,\}\right)$$



A hylomorphism, but it **recomputes**: exponential.

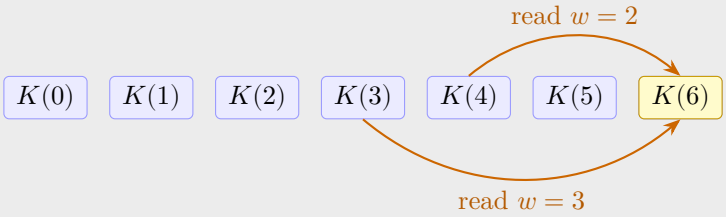
Remember, Don't Recompute



- **Dynamorphism**: a hylomorphism that keeps a *table* of earlier results.

R. Hinze, N. Wu, J. Gibbons. **Conjugate Hylomorphisms, or: The Mother of All Structured Recursion Schemes**. POPL 2015.

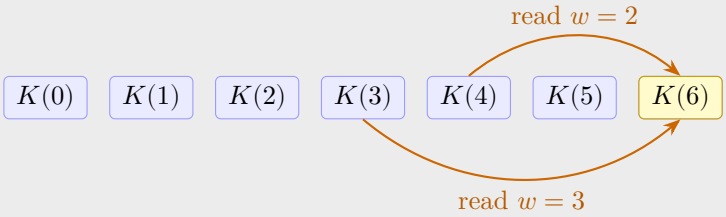
Remember, Don't Recompute



- **Dynamorphism**: a hylomorphism that keeps a *table* of earlier results.
- A law: recursion over *items* = recursion over *capacities* with a table, if each step *looks up* its subproblems.

R. Hinze, N. Wu, J. Gibbons. **Conjugate Hylomorphisms, or: The Mother of All Structured Recursion Schemes**. POPL 2015.

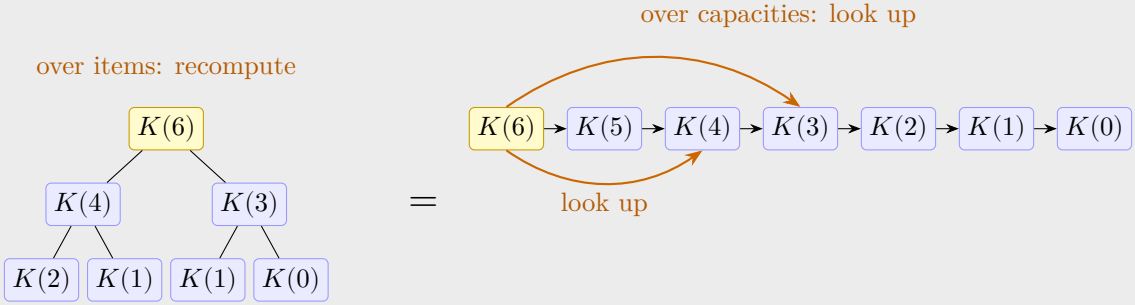
Remember, Don't Recompute



- **Dynamorphism**: a hylomorphism that keeps a *table* of earlier results.
- A law: recursion over *items* = recursion over *capacities* with a table, if each step *looks up* its subproblems.
- Exponential $\Longrightarrow$ one entry per capacity.

R. Hinze, N. Wu, J. Gibbons. **Conjugate Hylomorphisms, or: The Mother of All Structured Recursion Schemes**. POPL 2015.

From Recomputation to a Table, by Rewriting



From Recomputation to a Table, by Rewriting

```
Definition knapsack_naive wvs := hylo naive_alg (naive_c wvs).  (* exponential *)

(* Step 1: the algebra reads its subproblems through a lookup *)
Derive knap_alg_ in (forall g b, naive_alg (fmap g (naive_c wvs b))
                               = knap_alg_ (look_rec out_int_c g b))
  as knap_look.

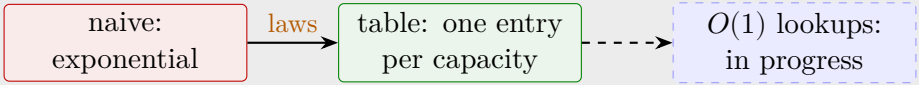
(* Step 2: recurse on capacities instead, tables in persistent arrays *)
Derive knapsack_arr in (knapsack_naive wvs =e knapsack_arr) as knapsack_arr_calc.
Proof.
  unfold knapsack_naive.
  rewrite (reenumerate (U := ATab int) (alg := knap_alg) knap_look).
  rewrite dyna_memo.
  finished.
Qed.
```

(simplified)

Knapsack: the Extracted Program

```
let rec look t j =                          (* j steps down the table *)
  if j = 0 then Some (head t)
  else if is_last t then None else look (rest t) (j - 1)
let best wvs t =                            (* t: results for c-1, c-2, ... *)
  max_int 0 (map_some (fun (w, v) ->
    if w = 0 then None else Option.map (fun u -> u + v) (look t (w - 1))) wvs)
let rec table wvs c =
  if c = 0 then single 0
  else let t = table wvs (c - 1) in push (best wvs t) t
let knapsack wvs c = head (table wvs c)
```

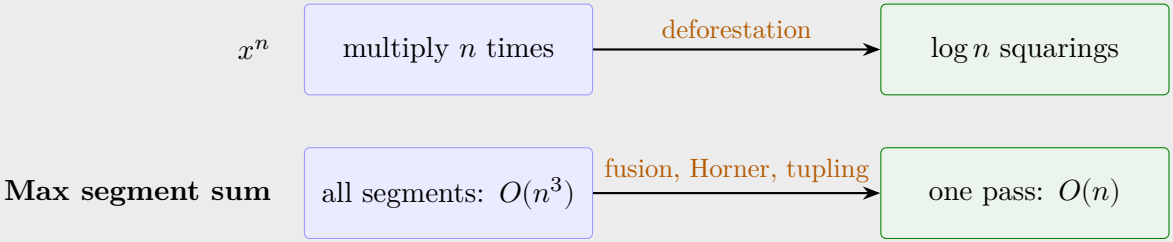
(simplified)



More Calculations

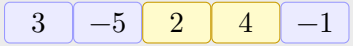


More Calculations



Maximum Segment Sum: Spec and Result

$$\mathsf{mss} = \mathsf{maximum} \circ \mathsf{map}\,\mathsf{sum} \circ \mathsf{segments}$$



best segment: $2 + 4 = 6$

Maximum Segment Sum: Spec and Result

$$\text{mss} = \text{maximum} \circ \text{map}\,\text{sum} \circ \text{segments}$$



best segment: $2 + 4 = 6$

```
let mss_code xs =
  let rec f = function
  | [] -> (Some 0, Some 0)
  | h :: t ->
    let (m, s) = f t in
    let s' = match s with
      | Some b -> Some (max 0 (h + b))
      | None -> Some 0 in
    (omax s' m, s')
  in fst (f xs)
```

(simplified)

Summary So Far

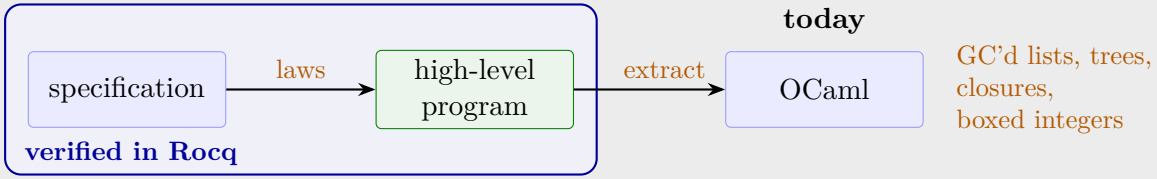
- A **library**: hylomorphisms and their laws in Rocq.
- Optimisations are **proofs by rewriting**.
- Termination proved **once per split**.
- **No extra axioms**; OCaml **without unsafe casts**.

D. Castro-Perez, M. Paviotti, M. Vollmer. **Program Optimisations via Hylomorphisms for Extraction of Executable Code**. ITP 2025. Library: `https://github.com/dcastrop/rocq-hylomorphisms`

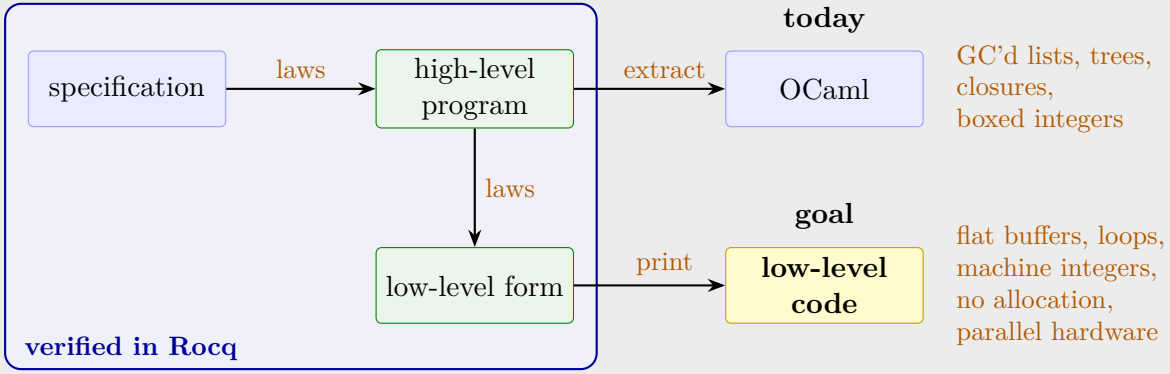
Work in Progress

Down to Low-Level Code

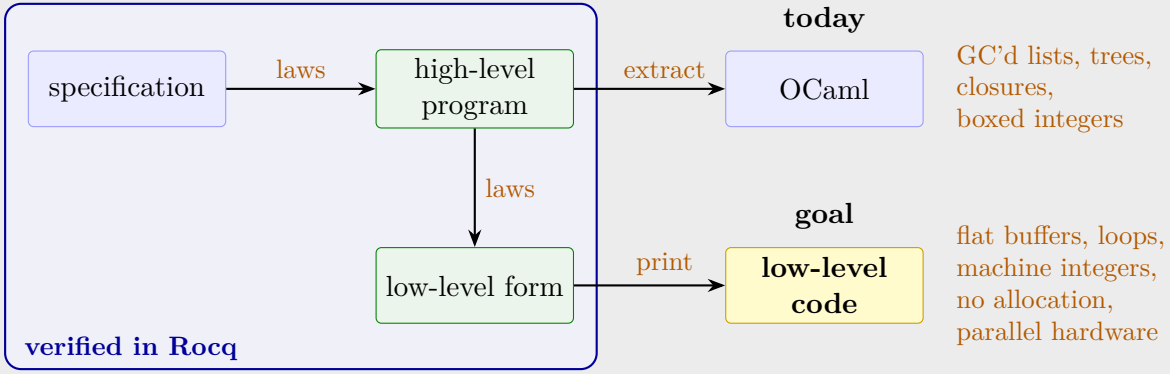
Two Ways Out of Rocq



Two Ways Out of Rocq



Two Ways Out of Rocq



Not "OCaml, then lower": **more calculation** reaches the low-level form; extraction is a direct print.

What We Are Working On

Low-level, **correct-by-construction** code,
extracted from **high-level calculation steps**,
possibly guided by a **cost model**.

- You choose *laws*, not pointers and indices.

What We Are Working On

Low-level, **correct-by-construction** code,
extracted from **high-level calculation steps**,
possibly guided by a **cost model**.

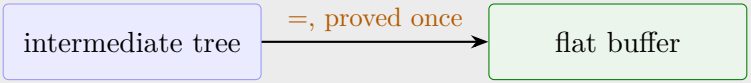
- You choose *laws*, not pointers and indices.
- Every step proved; the output is machine-level code.

What We Are Working On

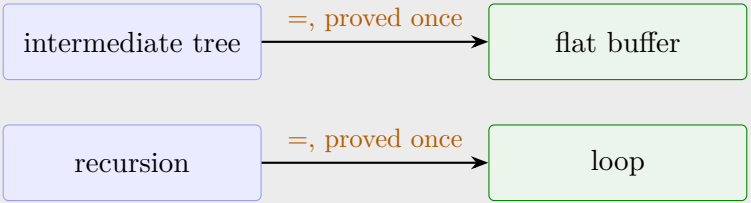
Low-level, **correct-by-construction** code,
extracted from **high-level calculation steps**,
possibly guided by a **cost model**.

- You choose *laws*, not pointers and indices.
- Every step proved; the output is machine-level code.
- A cost model says *which* equal program to pick.

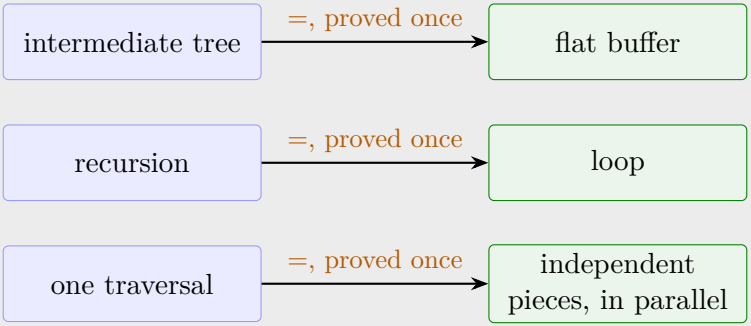
Ongoing: Laws That Change Representation



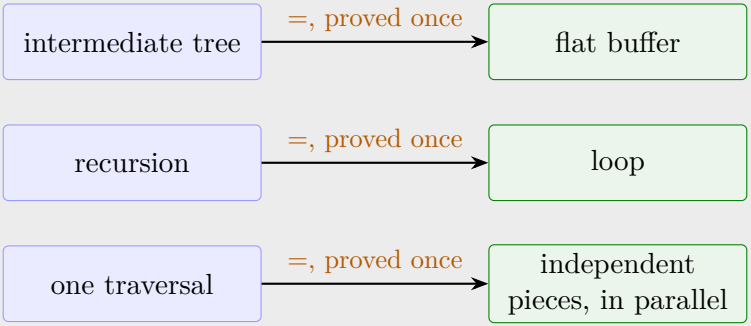
Ongoing: Laws That Change Representation



Ongoing: Laws That Change Representation

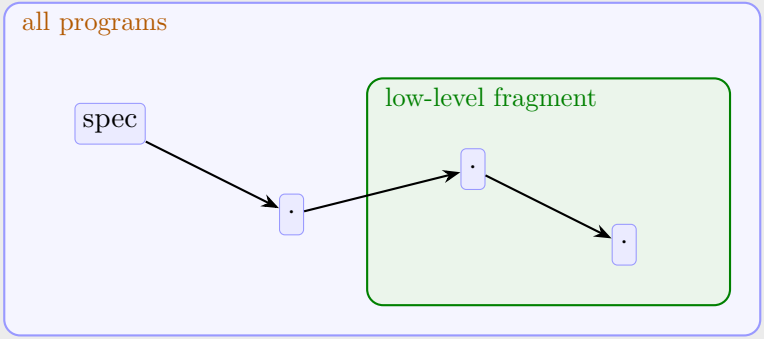


Ongoing: Laws That Change Representation



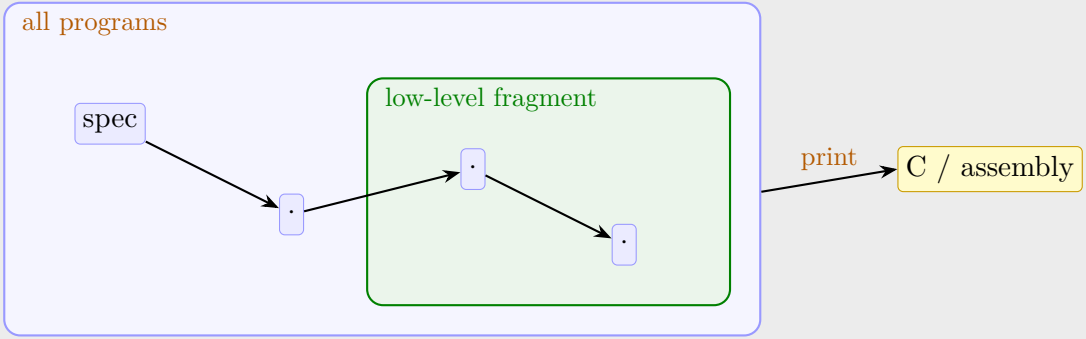
Today's laws change the *algorithm*; these change *where data lives* and *how work is scheduled*.

Ongoing: A Target the Laws Never Leave



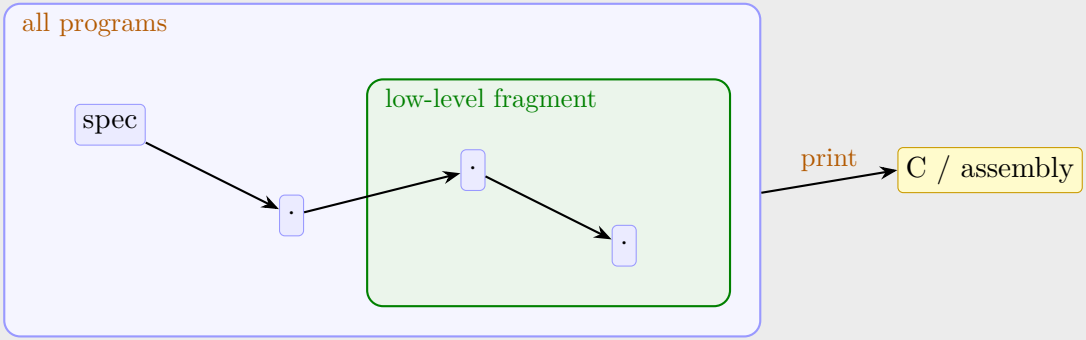
- Every law keeps programs **inside** the fragment.

Ongoing: A Target the Laws Never Leave



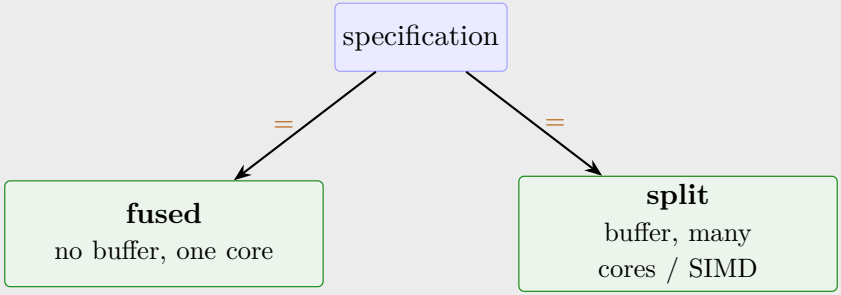
- Every law keeps programs **inside** the fragment.
- Only a **small printer** is trusted.

Ongoing: A Target the Laws Never Leave

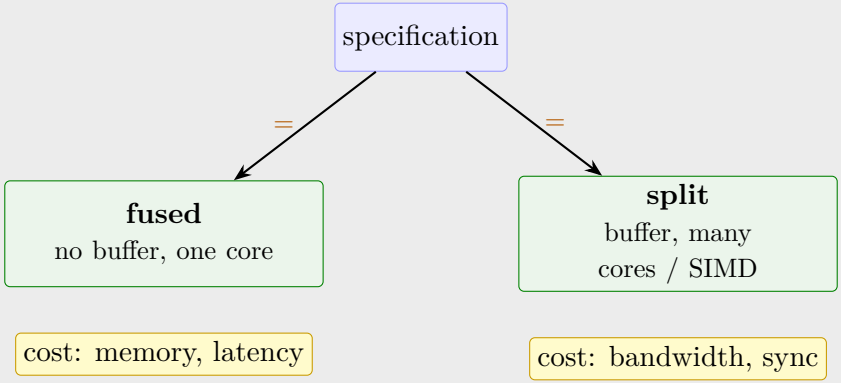


- Every law keeps programs **inside** the fragment.
- Only a **small printer** is trusted.
- Overflow and bounds: **side conditions** of the laws.

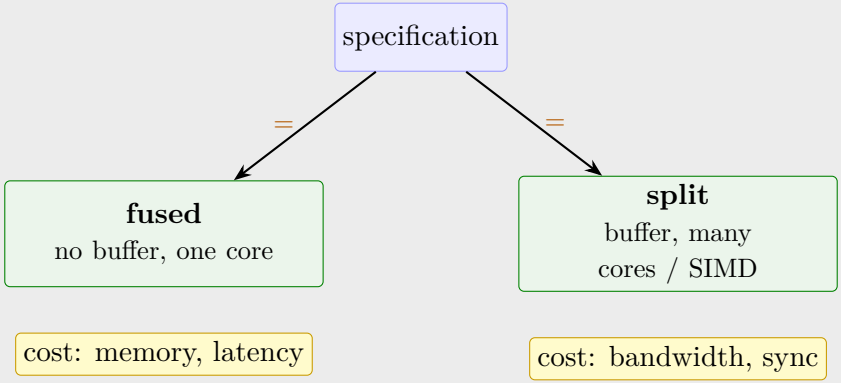
Ongoing: A Cost Model



Ongoing: A Cost Model

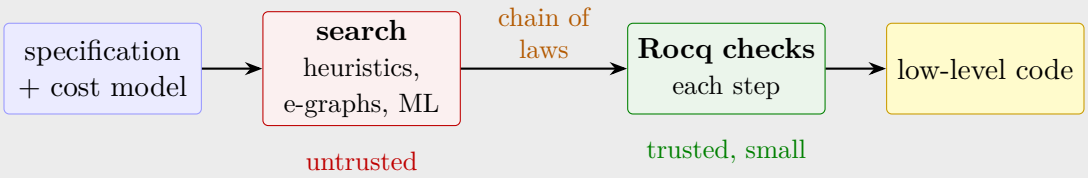


Ongoing: A Cost Model



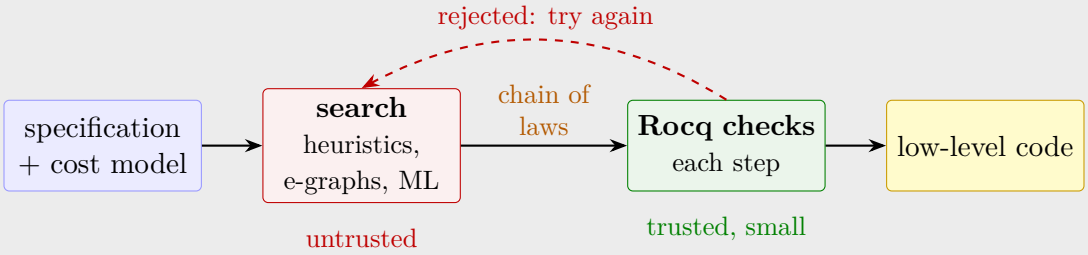
The laws say what is *equal*; a cost model says what is *better*.

Untrusted Search, Trusted Checking



- The derivation is a **certificate**.

Untrusted Search, Trusted Checking



- The derivation is a **certificate**.
- Search can be as clever, or as wrong, as it likes.

Where We Are



In Rocq, axiom-free:

- Splitting index ranges k ways
- Machine integers, no overflow (proved)
- Buffers, compared by their reads
- Tree reductions over ranges
- Writes to a buffer, driven by any split

Not yet:

- Laws that add or remove a buffer
- Each index written exactly once
- The low-level target and printer
- The cost model and search

Summary

- Program optimisation as **algebra**: rewrite with laws.
- Divide and conquer as **hylomorphisms**: laws proved once.
- In Rocq: **machine-checked**, no extra axioms, clean OCaml.
- Ongoing: **low-level, correct-by-construction code** from high-level steps, guided by a **cost model**.

Thank you!

Questions?

`https://github.com/dcastrop/rocq-hylomorphisms`

Based on joint work with Marco Paviotti and Michael Vollmer.